CORE ATTENDEE EDITION: Start with 00_START_HERE.md at the pack root. Only the core two-day route is supplied. Optional publishing, dummy website, resource creation, identity/networking and advanced Azure tracker extensions mentioned below are not assigned or included. Use your own extracted path where a command shows C:\YOUR-ATTENDEE-PACK. Do not copy instructor settings.

Application Insights - investigate a failed submission

LBS | Attendee edition | Fictional data only | Updated 28 October 2026

Before you start

Expected duration: 75 minutes: 11:30-12:45

Prerequisites: Prepared instrumented application and known request route supplied by the tutor, or fictional Day2Operations data.

Required permissions: Read access to the assigned Application Insights component and relevant logs/workspace, including approved table access. Instrumentation, connection settings and deployment changes are owner work, not core participant tasks.

Resources created: None; no alert or new telemetry resource required.

Cost considerations: Ingestion, retention, tests, exports and alerts may incur cost. Use a few approved requests, never a load test or unbounded debug logging.

Safety: Use the assigned tenant, subscription and resource only. Do not change shared networking, roles, tiers or deployment settings. No real documents, secrets or keyed URLs in evidence.

What, why and scope

Application Insights helps investigate an application's requests, failures and dependencies. A Log Analytics workspace can store related logs. Live Metrics and stored logs are different views and can update at different times.

Scenario: a fictional user reports a failed submission and another reports a slow page. Your task is to find request evidence and related dependencies, not to deliberately break the shared application.

Time split: explanation 10; tutor walkthrough 15; independent inspection 25; missing-data recovery 15; evidence and cost recap 10.

The supplied local tracker is not instrumented to Azure. A connection-string setting alone does not establish that application code is instrumented. Use the tutor's pre-tested app, not an assumed deployment of the tracker or the fixed-response Function.

Navigation and investigation

1. Confirm the app endpoint and Application Insights component with the tutor. In portal search Application Insights, open the assigned component, read its Overview and identify the linked workspace if visible.

Expected result: A known app/component/workspace relationship. Visibility of the component does not prove telemetry arrived.

2. Record current UTC time. Send at most five requests to the tutor-approved harmless endpoint, or observe the prepared traffic. Do not change database settings or provoke failure on a shared app.

Expected result: A bounded traffic record with endpoint, time and actual status. A request failure is recorded, not silently counted as success.

3. Open Investigate > Performance and Failures where shown; use portal menu search if the layout differs. Choose a recent window covering the test. Filter the correct role/app before drilling into an operation.

Expected result: Actual stored request rows or an explicit no-data condition. Ingestion may take several minutes; no universal arrival time is promised.

4. Open a request/sample and its end-to-end transaction details. Identify request name, result code, duration and operation/correlation ID. Inspect a dependency if instrumentation captured one.

Expected result: You distinguish request duration from dependency duration. A slow dependency is a lead, not proof of the complete root cause.

5. Open Logs, close Queries hub if present, switch Agent off and choose KQL mode. Confirm whether the schema lists AppRequests or requests.

Expected result: The actual editor and schema are identified for the KQL lab after lunch.

6. If live telemetry is unavailable, use Day2Operations\Queries\Prepared in an authorised editor, or review the fictional CSVs offline. Record the mode explicitly.

Expected result: You can practise investigation without claiming ingestion, instrumentation or live-cloud access passed.

Troubleshooting - symptom, checks and safe recovery

Symptom	Exact checks / recovery
No request rows	Check actual endpoint/time, app/component mapping, workspace scope, filters and instrumentation with the owner. Wait for ingestion, then retry a bounded read. Do not turn off security or add a second SDK.
Requests exist, dependencies absent	Confirm a dependency call actually occurred and that instrumentation supports it. The

	fixed-response Function does not call SQL. Empty dependencies do not prove SQL is healthy.
Table name not recognised	Inspect the editor schema. Workspace: AppRequests/AppDependencies. Application Insights resource: requests/dependencies. Use the matching supplied query directory.
Access denied	Record the component/workspace and non-secret error. Owner checks scoped query rights; do not substitute a master key.
Failure view empty	Use an approved known failure or prepared dataset. A 404 is not guaranteed to have the intended Success semantics. No fault injection required.

Knowledge check - before and after

Before: what can a successful /health response prove, and what can it not prove about SQL?

After: did the evidence come from live collection or prepared data? Which operation identifies the request? What would you check before changing instrumentation?

Cleanup

No core resources or alerts were created. Stop sending test requests and close any private credential view.

Keep only redacted fictional evidence. Do not delete retained logs or disable required monitoring.

Creating alerts, action groups or instrumentation is an optional owner-approved extension, not a required attendee step.

Your evidence

Record	Your result
Mode: live / prepared query / offline review	
Scope, time range and source	
Expected / actual result	
Passed / Blocked / Not run and why	
Cost/security trade-off	
Follow-up owner / cleanup	